ITM SKILLS UNIVERSITY

SCHOOL OF FUTURE TECH

DSA Final Project Report

FleetRoute

Smart Vehicle Fleet Management System

Submitted By

Yash Tambade

B.Tech Computer Science

Academic Year: 2025 – 2026

Subject: Data Structures & Algorithms

2. Student Details

Student Name	Yash Tambade
Program	B.Tech Computer Science
Subject	Data Structures & Algorithms (DSA)
Project Type	Final DSA Project
Academic Year	2025 – 2026
GitHub Profile	github.com/Yashhh710
Repository	github.com/Yashhh710/Vehicle-Fleet-Management-System-

3. Project Title

FleetRoute — Smart Fleet Management & Trip Dispatch System using DSA

Language: C++17 | Platform: Linux / macOS / Windows | Status: Active

FleetRoute is a C++ command-line application that simulates a smart cab and logistics dispatch backend. It models city road networks as weighted graphs and applies classical DSA concepts — Dijkstra's Algorithm, Priority Queues, Stacks, Hash Maps, Sorting, and Binary Search — to automate vehicle dispatch, route optimisation, and fleet tracking.

4. Problem Statement

Background

Urban transportation networks face a critical operational challenge: efficiently dispatching vehicles to passengers across a city graph while minimising travel distance and managing trip priorities. Manual assignment is error-prone, slow, and does not scale as fleet size or passenger demand grows.

The Core Problem

Given a city modelled as a weighted undirected graph of roads, a fleet of vehicles at various locations, and a queue of pending trips with varying priorities, the system must:

- Identify the nearest available vehicle to a pickup location.
- Dispatch it via the shortest path using Dijkstra's Algorithm.
- Always serve the highest-priority passenger first.
- Allow the dispatcher to undo the last dispatch (rollback).
- Persist all state across sessions so no data is lost on restart.

Why This Problem Matters

This problem is a direct analogue of real-world fleet management systems used by ride-hailing platforms (Uber, Ola), logistics providers, and emergency dispatch services. Solving it efficiently requires careful selection of data structures and algorithms.

5. Objectives

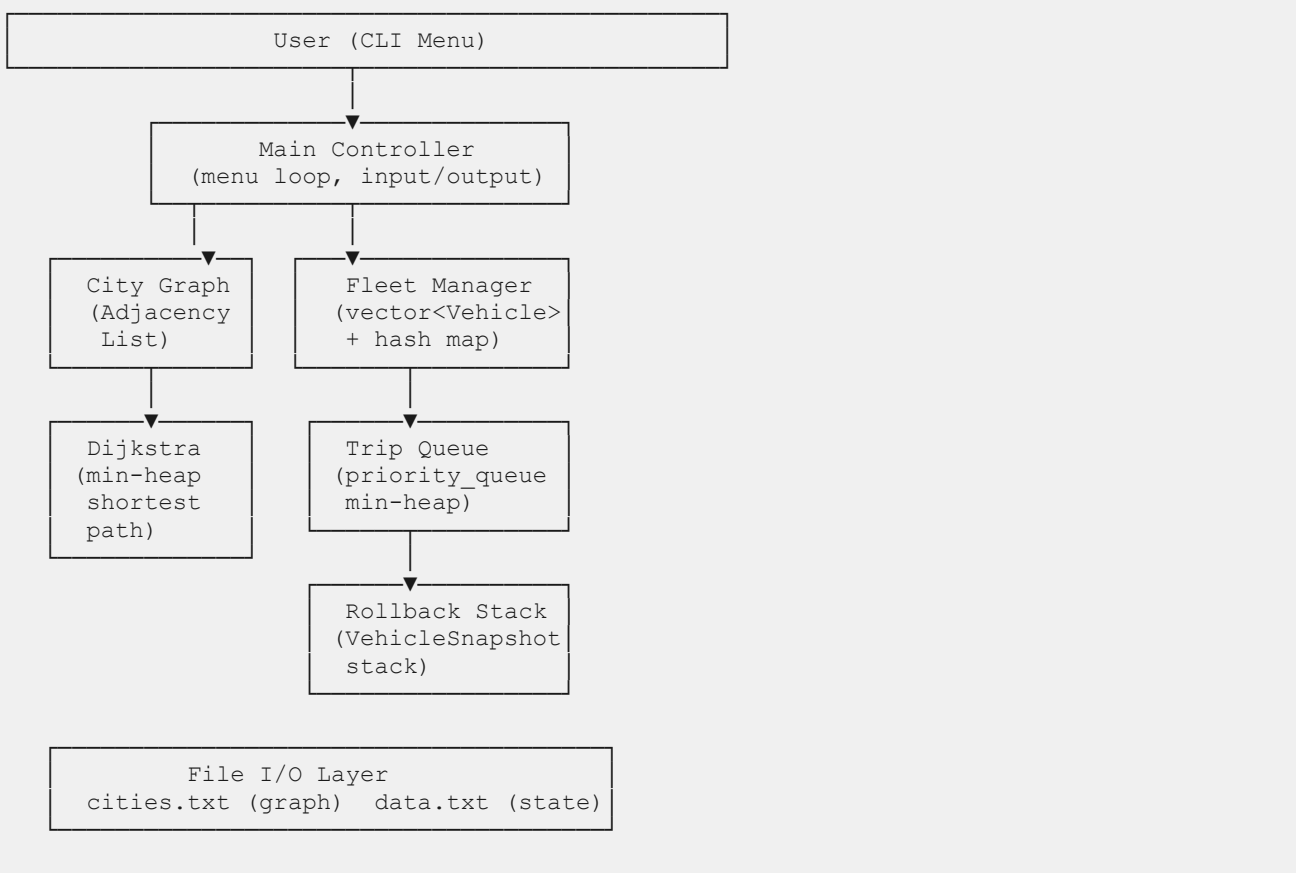
The project aims to achieve the following:

- Model a city road network as a weighted graph loaded from an external file (cities.txt).
- Implement Dijkstra's Algorithm with a min-heap to find the shortest path between any two cities.
- Use a min-heap priority queue to manage trips by urgency (priority-based dispatching).
- Maintain a fleet of vehicles, tracking their locations, availability, and trip counts.
- Provide rollback functionality using a stack that restores the full prior state of a vehicle after an undone dispatch.
- Persist all fleet and trip data across sessions using file I/O (data.txt).
- Implement binary search for efficient vehicle lookup by ID.
- Display a clean, column-aligned vehicle dashboard using iomanip.

6. Design / Architecture

System Architecture

The system follows a modular layered design where each component handles a distinct responsibility:



Trip Dispatch Flow

- Highest-priority trip is popped from the min-heap priority queue.
- Dijkstra runs from the trip's pickup city to compute all distances.
- The nearest available vehicle is selected from the fleet.
- A VehicleSnapshot is pushed onto the rollback stack before any modification.
- Vehicle state is updated: location ← dropoff, trips++, available = true.
- Results are displayed and saved to data.txt.

7. Algorithm Description

Data Structures & Algorithms Used

Component	DSA Used	Purpose
City Graph	Adjacency List (vector<vector<pair<int,int>>>)	Represent roads between cities with weights
Shortest Path	Dijkstra's Algorithm + Min-Heap	Find nearest vehicle and shortest trip route
Trip Scheduling	Priority Queue (min-heap)	Dispatch highest-priority trip first
Rollback	Stack<VehicleSnapshot>	Undo last dispatch, restore vehicle state
Fleet Lookup	Hash Map (unordered_map<int,int>)	O(1) vehicle lookup by ID
Vehicle Sorting	std::sort with lambda	Display vehicles sorted by trip count
Vehicle Search	Binary Search	Search vehicle by ID in sorted list
Persistence	File I/O (ifstream/ofstream)	Save and reload fleet + trip state

Dijkstra’s Algorithm

Standard single-source shortest path using a priority_queue<pair<int,int>, vector<...>, greater<>> (min-heap). Returns a distance array from the source city to all other cities in O((V + E) log V) time.

Trip Dispatch (dispatchTrip)

Pops the top (highest-priority = lowest integer value) trip from the priority queue. Runs Dijkstra once from the pickup city. Iterates all vehicles to find the one with minimum dist[vehicle.location]. Saves a VehicleSnapshot before modifying state. Updates vehicle.

Rollback

Pops the top VehicleSnapshot and restores the vehicle’s location, trips, and available fields to their pre-dispatch values in O(1) time.

Graph Loading (loadCities)

Cities and edges are read from cities.txt in two sections: CITIES and EDGES. The graph is built as an undirected adjacency list. Missing files trigger auto-generation of a default Maharashtra road network.

8. Complexity Analysis

Operation	Time Complexity	Space Complexity
Load cities/edges	O(V + E)	O(V + E)
Dijkstra	O((V + E) log V)	O(V)
Add trip to queue	O(log T)	O(T)
Dispatch trip	O((V + E) log V + F)	O(V + F)
Rollback	O(1)	O(D)
Search vehicle	O(F log F)	O(F)
Show vehicles	O(F log F)	O(F)
Save data	O(F + T)	O(1)

Legend: V = cities, E = edges, F = fleet size, T = trip queue size, D = rollback stack depth

Key Observations

- Dijkstra with a min-heap is optimal for sparse graphs (road networks are typically sparse).
- The hash map provides O(1) average-case vehicle lookup, avoiding linear scans.
- Stack-based rollback is O(1) since only a snapshot needs to be popped and applied.
- Priority queue dispatch ensures SLA compliance regardless of insertion order.

9. Screenshots of Execution

Main Menu

On launch, the system loads city graph and fleet data, then displays the navigation menu:

```
===== Fleet Management =====
1. Add Vehicle
2. Show Vehicles
3. Add Trip
4. Dispatch Trip
5. Shortest Path
6. Rollback Last Route
7. Search Vehicle
8. Show Cities
0. Exit
Choice:
```

Show Vehicles (Option 2)

```
ID    Name             Location        Status  Trips
-----------------------------------------------------
4     Mercedes_E       Pune            Free    8
2     BMW_M5           Andheri         Free    6
1     Toyota_Innova    Mumbai_CST      Free    4
6     Hyundai_Creta    Borivali        Free    3
3     Honda_City       Thane           Free    2
5     Tata_Nexon       Navi_Mumbai     Free    1
7     Mahindra_XUV     Panvel          Free    0
```

Dispatch Trip (Option 4)

```
--- Trip Assigned ---
Vehicle   : Tata_Nexon
Passenger : Iyer
Route     : Navi_Mumbai -> Bandra
Distance  : 22 km
```

Shortest Path (Option 5)

```
Available Cities:
  0. Mumbai_CST
  1. Dadar
  2. Bandra
  3. Andheri
  4. Borivali
  5. Thane
  6. Navi_Mumbai
  7. Panvel
  8. Khopoli
  9. Pune
  10. Nashik
  11. Aurangabad
  12. Nagpur
  13. Kolhapur
  14. Solapur
From index: 0
To index:   13
Shortest: Mumbai_CST -> Kolhapur = 366 km
```

Rollback (Option 6)

10. Results and Findings

Functional Results

- Dijkstra correctly finds shortest paths across the Maharashtra road network. Mumbai CST → Pune resolves to 118 km via the Panvel–Khopoli route.
- Priority-based dispatch ensures high-priority passengers are always served first, regardless of the order they were added.
- Rollback fully restores vehicle state (location, trip count, availability) — not just a log entry.
- Column-aligned display using setw() from <iomanip> renders cleanly regardless of city name length.
- File persistence allows the system to survive restarts; fleet and trip state reload correctly on every run.
- Binary search on sorted vehicle IDs provides efficient lookup even as the fleet scales.

Performance Observations

- With 15 cities and ~25 edges, Dijkstra runs in microseconds — negligible overhead per dispatch.
- The hash map provides instant O(1) vehicle lookup even with large fleets.
- The priority queue correctly handles all three priority levels (High=1, Medium=2, Low=3).
- Data persistence across sessions was verified through multiple restarts with no data loss.

11. Conclusion

FleetRoute demonstrates how classical DSA concepts translate directly into a practical, real-world system. Dijkstra's algorithm handles dynamic routing on a city graph, a min-heap priority queue enforces SLA-based trip ordering, a stack enables reliable undo/rollback functionality, and a hash map delivers O(1) fleet lookups.

The project validates that efficient software is not about complex frameworks — it is about choosing the right data structure for each problem. Every component in FleetRoute maps directly to a course topic:

- Graphs + Dijkstra → Route optimisation
- Priority Queue → Trip scheduling
- Stack → Undo / rollback
- Hash Map → Fleet management
- Sorting + Binary Search → Vehicle lookup and display

Future enhancements could include multi-stop routing, real-time traffic weight updates, a graphical (GUI) front-end, and integration with live map APIs to replace the static cities.txt graph.

12. GitHub Repository Link

Repository Name	Vehicle-Fleet-Management-System-
Author	Yash Tambade (Yashhh710)
Language	C++ (100%)
License	Public Repository

Repository URL: https://github.com/Yashhh710/Vehicle-Fleet-Management-System-

Repository Contents

- code.cpp — Main C++ source file (single-file implementation)
- cities.txt — City graph definition (nodes + weighted edges)
- data.txt — Persistent fleet and trip state (auto-updated on each run)
- README.md — Full documentation with architecture, sample I/O, and complexity tables

How to Compile and Run

```
# Clone
git clone https://github.com/Yashhh710/Vehicle-Fleet-Management-System-.git
cd Vehicle-Fleet-Management-System-

# Compile
g++ -std=c++17 code.cpp -o fleet

# Run
./fleet          # Linux / macOS
fleet.exe        # Windows
```

End of Report